Experimento: MQTT vs HTTPS para telemetría con ESP32 en QEMU

Oct 23, 2026 · @Juan Ignacio Cleri

Idea general

Las mismas 8 ESP32 en QEMU envían telemetría periódica a un servidor por dos caminos: MQTT a través de un broker, y HTTPS con un POST por lectura. El contenido de cada lectura es el mismo en ambos caminos y se serializa en JSON o en Protocol Buffers. Así se pueden separar dos efectos: el del transporte y el de la serialización.

A diferencia del experimento de luciérnagas, el caso de uso es neutral: enviar lecturas de los nodos al servidor es algo que ambas arquitecturas hacen de forma razonable.

Preguntas de investigación

1. ¿Cuántos bytes cuesta enviar una lectura por MQTT y por HTTPS, contando cabeceras y handshakes?
2. ¿Cómo se comparan la latencia extremo a extremo y la tasa de entrega de ambos?
3. ¿Cómo se degradan ambos protocolos con pérdida y latencia en la red?
4. ¿Qué parte de la diferencia viene de la conexión persistente y qué parte de TLS?
5. ¿Cuánto cambia el costo al pasar de JSON a protobuf en cada transporte, y cómo depende ese ahorro del tamaño y la forma del mensaje?

Estas preguntas cubren los objetivos de la monografía que comparan MQTT con HTTP y que relacionan MQTT con la capa de transporte (TCP, keep-alive y TLS).

Fundamento

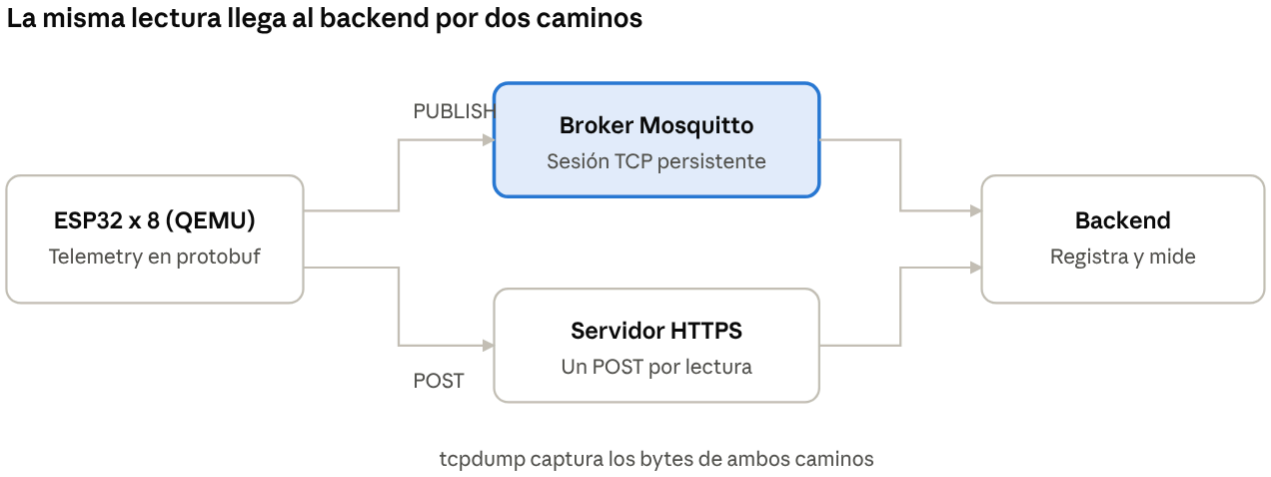
La diferencia principal no está en el payload sino en cómo cada protocolo usa la conexión TCP.

Aspecto	MQTT	HTTPS
Conexión	Una sesión TCP persistente por nodo	Una conexión por request, o reutilizada con keep-alive
Handshake TLS	Una vez, al conectar	En cada conexión nueva

Aspecto	MQTT	HTTPS
Cabecera por mensaje	Mínimo 2 bytes, más el tópico	Headers HTTP en texto, varios cientos de bytes
Respuesta	PUBACK solo con QoS 1 o 2	Siempre una respuesta HTTP
Garantía de entrega	QoS 0, 1 o 2 definidos por el protocolo	Reintentos implementados por la aplicación
Iniciativa	El servidor puede enviar al nodo por la misma sesión	Solo el cliente inicia

Por eso se prueban dos variantes de HTTPS: una conexión por request y una conexión reutilizada. La segunda es la comparación justa, y la primera muestra el peor caso que se da en dispositivos que duermen entre envíos.

Arquitectura



arquitectura · MQTT y HTTPS hacia el mismo backend

En cada corrida se usa un solo camino. El backend recibe las lecturas de los dos con el mismo código de registro, así que la medición es comparable.

Mensaje, tópicos y endpoints

El mismo contenido se envía en dos formatos. Es borrador. El campo repetido permite variar el tamaño del mensaje sin cambiar el esquema.

```
syntax = "proto3";

message Telemetry {
  uint32 esp_id          = 1;
  uint32 seq             = 2;
  uint64 t_us            = 3;
  uint32 status          = 4;
  repeated float values  = 5;
}
```

Equivalente en JSON compacto, sin espacios:

```
{"esp_id":5,"seq":142,"t_us":1729690000000000,"status":0,"values":[23.4]}
```

Camino	Destino	Método	Cuerpo con protobuf	Cuerpo con JSON
MQTT	Tópico exp/tel/{esp}/pb o exp/tel/{esp}/json	PUBLISH con QoS 0 o 1	Telemetry en binario	Texto JSON
HTTPS	Endpoint /tel/{esp}	POST	Content-Type application/x-protobuf	Content-Type application/json

En ambos casos {esp} es el número de nodo. En MQTT el formato va en el tópico, porque MQTT no tiene un campo de tipo de contenido como HTTP (salvo en MQTT 5).

Configuraciones y repeticiones

El núcleo del experimento es una matriz de transporte por serialización: 4 transportes por 2 formatos dan 8 combinaciones. Todas se corren con la base: 8 ESP, período de 1 s, TLS activado, pérdida 0 %, latencia 0 ms y mensaje de 1 valor.

Transporte	JSON	Protobuf
MQTT QoS 0	Combinación 1	Combinación 2
MQTT QoS 1	Combinación 3	Combinación 4
HTTPS reutilizado	Combinación 5	Combinación 6
HTTPS por request	Combinación 7	Combinación 8

Luego se varía un factor a la vez. Para no multiplicar las corridas, esas variaciones se aplican a MQTT QoS 1 y HTTPS reutilizado, cada uno con ambos formatos.

Factor	Valores a probar	Pregunta que responde
Valores por mensaje	1, 10, 50	¿Cómo cambia el ahorro de protobuf con el tamaño del mensaje?
Tipo de dato	Enteros chicos, floats, texto corto	¿Para qué datos protobuf ahorra más?
Nodos ESP	2, 4, 8	¿Cómo escala la carga del servidor?
Período de envío	100 ms, 1 s, 10 s	¿Cuándo pesa el handshake?
TLS	Con y sin TLS	¿Qué parte del costo es cifrado?
Pérdida	0 %, 1 %, 5 %	¿Cómo se degrada la entrega?
Latencia	0 ms, 50 ms, 200 ms	¿Cómo afecta a cada handshake?

Cada configuración se ejecuta 5 veces, y se reportan media y desvío.

Métricas

Los bytes se miden en la red, no en la aplicación, para incluir cabeceras TCP, TLS y del protocolo.

Métrica	Cómo se mide	Qué permite concluir
Bytes por lectura	Captura con tcpdump, total de bytes / lecturas entregadas	Costo real de ancho de banda
Bytes de payload	Tamaño del cuerpo serializado, en JSON y en protobuf	Ahorro de la serialización sola
Proporción payload / total	Bytes de payload / bytes por lectura	Cuánto pesa la serialización frente al transporte
Tiempo de serialización	Contador de ciclos en el firmware, al codificar y al decodificar	Costo de CPU de cada formato
Bytes de handshake	Bytes antes del primer dato en cada conexión	Peso de TCP y TLS

Métrica	Cómo se mide	Qué permite concluir
Paquetes por lectura	Captura con tcpdump	Overhead de acuses y respuestas
Latencia extremo a extremo	Recepción en el servidor − t_us	Rapidez de entrega
Tasa de entrega	Lecturas recibidas / enviadas, por seq	Efecto de la pérdida
Tiempo de conexión	Desde el SYN hasta el fin del handshake TLS	Costo de abrir conexiones
Factor de tiempo real	Tiempo simulado / tiempo del host	Validez de la corrida

El número de secuencia permite detectar pérdidas y duplicados, que son esperables con MQTT QoS 1.

Procedimiento de cada corrida

1. Levantar el broker Mosquitto y el servidor HTTPS, ambos con el mismo certificado de prueba.
2. Levantar el backend que recibe por los dos caminos y registra cada lectura.
3. Iniciar la captura con tcpdump en la interfaz que usan las instancias.
4. Levantar las instancias QEMU con el transporte de la corrida.
5. Enviar lecturas durante 120 s.
6. Detener la captura y guardar mensajes, métricas y configuración en CSV o JSON.
7. Repetir con cada variante de la sección de configuraciones.

Los dos transportes nunca corren a la vez, para que no compitan por CPU ni por la red.

Limitaciones

- El handshake TLS en QEMU es más lento que en un ESP32 real, porque la criptografía se emula por software. Los tiempos de conexión con TLS se reportan como relativos, no absolutos.
- QEMU no emula WiFi, así que la red es Ethernet emulado con condiciones inyectadas por tc netem.
- Solo se mide telemetría del nodo al servidor. El envío de comandos del servidor al nodo, donde HTTP necesita polling, queda fuera de esta comparación.

- El servidor HTTPS y el broker son implementaciones concretas. Otras implementaciones pueden dar números distintos.
- Las 8 instancias comparten la CPU del host, por eso se controla el factor de tiempo real.

Hipótesis orientativas

- MQTT envía menos bytes por lectura que HTTPS, y la diferencia crece cuando el período es corto.
- HTTPS con conexión reutilizada reduce gran parte de la diferencia, pero los headers HTTP siguen pesando más que la cabecera MQTT.
- HTTPS con una conexión por request es el caso más costoso, sobre todo con TLS y latencia alta, porque cada handshake suma idas y vueltas.
- Con pérdida, MQTT QoS 1 mantiene la entrega a costa de duplicados, y MQTT QoS 0 pierde lecturas.
- Protobuf reduce el payload en ambos caminos, pero su efecto sobre el total es mayor en MQTT, donde el payload es la mayor parte del mensaje. En HTTPS, los headers diluyen el ahorro.
- El ahorro de protobuf crece con la cantidad de valores numéricos, porque JSON repite los nombres de campo y escribe los números como texto. Con texto corto, la diferencia debería ser menor.
- El tiempo de serialización en el nodo es menor con protobuf (nanopb) que con una librería JSON, aunque la diferencia es chica frente a la latencia de red.

Estas hipótesis sirven tanto para confirmarse como para descartarse.